

Unidad Didáctica

**Estructuras Algorítmicas de Control,
Unidad III**

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	5
Estructuras de control	5
Estructura algorítmica Secuencia	5
Estructuras algorítmicas Selectivas.....	6
Estructuras algorítmicas Repetitivas	10
Bibliografía y Webgrafía	21
Glosario	22



En la tercera unidad de nuestro curso, nos adentraremos en las Estructuras Algorítmicas de Control, un conjunto fundamental de herramientas que nos permiten dirigir el flujo de ejecución de un programa de manera eficiente y efectiva. En este módulo, nuestro enfoque estará en comprender y aplicar estas estructuras para resolver problemas simples, así como para

abordar situaciones más complejas que requieran expresiones condicionales y cíclicas.

Nuestros objetivos específicos en esta unidad incluyen la resolución de problemas mediante el uso de estructuras básicas de control, la aplicación adecuada de expresiones condicionales y bucles en la solución de problemas, así como la familiarización con el uso de editores e intérpretes de algoritmos representados en diagramas de flujo y entornos integrados con compiladores de lenguaje algorítmico. Además, promoveremos la adquisición de hábitos de respeto hacia los derechos de autor en el ámbito de la programación.

Los contenidos que abordaremos en esta unidad incluyen las estructuras de control secuenciales, selectivas y repetitivas. Exploraremos detalladamente las estructuras selectivas, desde las alternativas simples hasta las múltiples, así como las anidadas. Del mismo modo, nos sumergiremos en las estructuras repetitivas, comprendiendo la importancia de los contadores, acumuladores y las distintas formas de

implementar bucles, tales como el for, while-do y do-while, así como la gestión de salidas internas y estructuras repetitivas anidadas.

A lo largo de este viaje, fortaleceremos nuestras habilidades para desarrollar algoritmos eficientes y robustos, sentando las bases para un dominio sólido de la programación estructurada y la resolución de problemas computacionales. ¡Prepárate para explorar el fascinante mundo de las estructuras algorítmicas de control!

Estructuras de control

Estructura algorítmica Secuencia

La estructura secuencial es fundamental en la programación y consiste en una secuencia de pasos que se ejecutan uno tras otro. En esta unidad, exploraremos cómo utilizar esta estructura en PSeInt para desarrollar algoritmos paso a paso. Comprenderemos que esta estructura sigue una secuencia lógica de pasos, ejecutándolos en el orden en que están escritos.

Para comenzar, debemos declarar todas las variables que necesitaremos en nuestro algoritmo. Las variables son contenedores que almacenan valores y pueden cambiar durante la ejecución del programa. Siguiendo las convenciones de PSeInt, todas las variables deben ser declaradas y todas las palabras en los identificadores deben estar en minúsculas, separadas por guiones bajos. Por ejemplo:

```
Algoritmo ejemplo_secuencia
    definir numero1, numero2, suma como entero;
    numero1 = 5;
    numero2 = 3;
    suma = 0;
```

Aquí, hemos definido tres variables: numero1, numero2 y suma, todas del tipo entero. Luego, hemos inicializado numero1 con el valor 5, numero2 con el valor 3 y suma con el valor 0.

Secuencia de Instrucciones: Después de la declaración de variables, seguimos con la secuencia de instrucciones. Estas instrucciones se ejecutan en el orden en que están escritas, una después de la otra. Por ejemplo:

```
    suma = numero1 + numero2;
    escribir "La suma es: ", suma;
FinAlgoritmo
```

Aquí, hemos realizado dos acciones en secuencia: primero, hemos sumado numero1 y numero2, y luego hemos mostrado el resultado por pantalla utilizando la instrucción escribir.

Ejemplo Completo: Ahora, veamos un ejemplo completo de un algoritmo que utiliza la estructura secuencial para calcular el área de un rectángulo:

```
Algoritmo calcular_area_rectangulo
    definir base, altura, area como real;
    base = 10;
    altura = 5;
    area = 0;

    area = base * altura;

    escribir "El área del rectángulo es: ", area;
FinAlgoritmo
```

Aquí, hemos declarado las variables base, altura y area como números reales. Luego, asignamos valores a base y altura. Después, calculamos el área del rectángulo multiplicando la base por la altura y mostramos el resultado por pantalla.

Estructuras algorítmicas Selectivas

Introducción

Las estructuras selectivas son herramientas poderosas en la programación que permiten tomar decisiones basadas en condiciones específicas. En esta unidad, exploraremos varias formas de estructuras selectivas en PSeInt y cómo utilizarlas para controlar el flujo de ejecución de un programa según las condiciones establecidas.

Alternativa simple (si-entonces / if-then)

La alternativa simple es la forma más básica de una estructura selectiva. Se utiliza para ejecutar un conjunto de instrucciones si se cumple una condición determinada.

Por ejemplo:

```
Algoritmo ejemplo_alternativa_simple
  Definir edad como entero;
  edad = 18;
  Si edad >= 18 Entonces
    Escribir "Eres mayor de edad";
  FinSi
FinAlgoritmo
```

En este ejemplo, si la variable edad es igual o mayor que 18, se imprimirá "Eres mayor de edad".

Alternativa doble (si-entonces-sino / if-then-else)

La alternativa doble permite ejecutar un conjunto de instrucciones si se cumple una condición y otro conjunto de instrucciones si no se cumple. Por ejemplo:

```
Algoritmo ejemplo_alternativa_doble
  Definir numero como entero;
  numero = 10;
  Si numero < 0 Entonces
    Escribir "El número es negativo";
  Sino
    Escribir "El número es positivo o cero";
  FinSi
FinAlgoritmo
```

En este caso, si numero es menor que 0, se imprimirá "El número es negativo"; de lo contrario, se imprimirá "El número es positivo o cero".

Alternativa múltiple (según —sea / caso de — case)

La alternativa múltiple se utiliza cuando hay múltiples condiciones posibles a evaluar. Cada condición se evalúa en una estructura caso y se ejecuta el bloque de código correspondiente al primer caso que se cumple. Por ejemplo:

```
Algoritmo ejemplo_alternativa_multiple
  Definir dia_semana como entero;
  dia_semana = 3;
  Segun dia_semana Sea
    1: Escribir "Lunes";
    2: Escribir "Martes";
    3: Escribir "Miércoles";
    4: Escribir "Jueves";
    5: Escribir "Viernes";
    6: Escribir "Sábado";
    7: Escribir "Domingo";
  FinSegun
FinAlgoritmo
```

En este ejemplo, se imprimirá el día de la semana correspondiente al valor de dia_semana.

Estructura selectiva en cascada (anidadas)

Las estructuras selectivas también pueden estar anidadas, lo que significa que pueden contener otras estructuras selectivas dentro de ellas. Esto se utiliza cuando las decisiones dependen de múltiples condiciones. Por ejemplo:

```
Algoritmo ejemplo_selectiva_anidada
  Definir nota as Real;
  nota = 12;
  Si nota >= 10 Entonces
    Si nota < 14 Entonces
      Escribir "Aprobado";
    Sino
      Escribir "Sobresaliente";
    FinSi
  Sino
    Escribir "Suspenso";
  FinSi
FinAlgoritmo
```

En este ejemplo, se evalúa primero si la nota es mayor o igual a 10, y luego, dentro de ese caso, se evalúa si la nota es menor que 14 para determinar si es "Aprobado" o "Sobresaliente".

Formato y funcionamiento de las selectivas

Es importante entender el formato y el funcionamiento de las estructuras selectivas para escribir algoritmos claros y eficientes. Cada estructura selectiva debe comenzar con un si seguido de la condición a evaluar, y terminar con un finSi. En el caso de las estructuras si-entonces-sino y según —sea, se utiliza sino o caso contrario para manejar los casos que no cumplen la condición inicial.

Dominar estas estructuras selectivas es fundamental para desarrollar habilidades sólidas de programación y para resolver una amplia variedad de problemas de manera eficiente y efectiva en PSeInt.

```
Algoritmo numeros_complejos
    Definir num1, num2 como Entero;

    Escribir "Ingrese el primer número:";
    Leer num1;
    Escribir "Ingrese el segundo número:";
    Leer num2;

    Si (num1 > 10 Y num2 > 10) O (num1 = 5 O num2 = 5) Entonces
        Escribir "¡Al menos uno de los números es mayor que 10 o uno de los
números es igual a 5!";
    Sino
        Escribir "Los números no cumplen las condiciones requeridas.";
    FinSi
FinAlgoritmo
```

Estructuras algorítmicas Repetitivas

En esta clase, exploraremos las estructuras algorítmicas repetitivas, también conocidas como bucles, y las asignaciones especiales, que incluyen el uso de contadores y acumuladores. Estas herramientas son fundamentales en la programación para realizar tareas repetitivas y llevar un seguimiento de ciertos valores mientras se ejecuta un algoritmo. Utilizaremos PSeInt como herramienta para comprender y aplicar estos conceptos con ejemplos claros y fáciles de entender

Asignaciones especiales

Antes de profundizar en las estructuras repetitivas, es importante comprender las asignaciones especiales, que nos permiten llevar un control de la cantidad de veces que se ejecuta un bucle (contador) o acumular valores a lo largo del proceso (acumulador). Veamos un ejemplo:

Algoritmo asignaciones_especiales

 Definir contador, acumulador, numero como Entero;

 contador = 0;

 acumulador = 0;

 Mientras contador < 5 Hacer

 Escribir "Ingrese un número:";

 Leer numero;

 acumulador = acumulador + numero;

 contador = contador + 1;

 FinMientras

 Escribir "La suma de los números ingresados es:", acumulador;

FinAlgoritmo

En este ejemplo, utilizamos un contador para llevar un registro de cuántas veces se ejecuta el bucle y un acumulador para sumar los números ingresados por el usuario. Observa cómo se incrementan tanto el contador como el acumulador dentro del bucle.

Contador

Los contadores son variables especialmente designadas para llevar un registro del número de veces que se ha ejecutado un bucle o de la ocurrencia de ciertos eventos en un programa. Son una herramienta fundamental en la programación, ya que nos permiten controlar y gestionar la repetición de tareas de manera eficiente.

- Un contador generalmente se inicializa antes de que comience un bucle.
- Dentro del bucle, el contador se incrementa o modifica según sea necesario para rastrear el progreso del bucle.
- El contador se utiliza en la condición de salida del bucle para determinar cuándo detener la ejecución del mismo.

Beneficios de los Contadores:

- Control de Iteraciones: Permiten ejecutar un bucle un número específico de veces.
- Seguimiento de Eventos: Se utilizan para contar eventos específicos dentro de un programa.
- Medición de Rendimiento: Ayudan a evaluar el rendimiento del programa al contar el número de operaciones realizadas.

Casos de Uso:

- Iteración Controlada: Cuando se necesita realizar una tarea un número específico de veces, como imprimir una secuencia de números o realizar cálculos repetitivos.
- Recuento de Elementos: Para contar la cantidad de elementos en una lista o matriz.
- Seguimiento de Acciones: Para llevar un registro de las acciones realizadas por el usuario en una aplicación.
- Validación de Entradas: Para verificar si se han ingresado cierta cantidad de datos requeridos antes de continuar con una operación.

Ejemplo de Uso de Contador:

Supongamos que queremos imprimir los números del 1 al 10. Podemos usar un contador para controlar la iteración del bucle y asegurarnos de que se impriman los números correctos.

```
Algoritmo ejemplo_contador
  Definir contador como Entero;

  contador = 1; // Inicializamos el contador

  Mientras contador <= 10 Hacer
    Escribir contador;
    contador = contador + 1; // Incrementamos el contador
  FinMientras
FinAlgoritmo
```

En este ejemplo, el contador se inicializa en 1 antes de que comience el bucle. Luego, dentro del bucle, se imprime el valor del contador y se incrementa en 1 en cada iteración. El bucle se detiene cuando el contador alcanza o supera el valor de 10. De esta manera, el contador nos permite controlar la impresión de los números del 1 al 10 de manera eficiente.

Acumulador

Los acumuladores son variables que se utilizan para almacenar y acumular valores a lo largo de la ejecución de un programa. Son útiles cuando necesitamos sumar, contar o calcular valores durante un proceso repetitivo, como en bucles, y nos permiten mantener un registro de información importante mientras el programa se ejecuta.

- Un acumulador generalmente se inicializa antes de que comience un proceso repetitivo, como un bucle.
- Dentro del proceso, el acumulador se actualiza según sea necesario, ya sea agregando nuevos valores, sumando cantidades, concatenando cadenas, etc.

- Al final del proceso, el valor acumulado en el acumulador se utiliza según los requisitos del programa.

Beneficios de los Acumuladores:

- Suma de Valores: Se utilizan para sumar una serie de valores durante un proceso repetitivo.
- Conteo de Elementos: Pueden contar el número de veces que ocurre un evento específico durante la ejecución de un programa.
- Almacenamiento de Resultados Parciales: Permiten almacenar resultados parciales mientras se realiza una serie de operaciones.

Casos de Uso:

- Suma de Números: Para calcular la suma de una lista de números.
- Conteo de Elementos: Para contar la cantidad de elementos que cumplen cierta condición en una lista o matriz.
- Promedio de Valores: Para calcular el promedio de un conjunto de datos.
- Concatenación de Cadenas: Para unir múltiples cadenas en una sola cadena más larga.
- Registro de Total de Ventas: Para mantener un registro del total de ventas en una tienda durante un período de tiempo específico.

Ejemplo de Uso de Acumulador:

Supongamos que queremos calcular la suma de los primeros 10 números naturales. Podemos usar un acumulador para almacenar la suma de los números a medida que los calculamos en un bucle.

```

Algoritmo ejemplo_acumulador
  Definir contador, suma como Entero;

  contador = 1; // Inicializamos el contador
  suma = 0;     // Inicializamos el acumulador

  Mientras contador <= 10 Hacer
    suma = suma + contador; // Sumamos el valor actual del contador al
    acumulador
    contador = contador + 1; // Incrementamos el contador
  FinMientras

  Escribir "La suma de los primeros 10 números naturales es:", suma;
FinAlgoritmo

```

En este ejemplo, el acumulador suma se inicializa en 0 antes de que comience el bucle. Luego, dentro del bucle, se suma el valor actual del contador al acumulador en cada iteración. El bucle se detiene cuando el contador alcanza o supera el valor de 10. Al final, el valor acumulado en el acumulador suma representa la suma de los primeros 10 números naturales.

Introducción.

Ahora, exploraremos las tres estructuras repetitivas principales: for, while-do y do-while. Estas estructuras nos permiten repetir un bloque de código mientras se cumpla una condición específica. Veamos ejemplos de cada una:

Estructura repetitiva for (desde/para).

La estructura repetitiva for, también conocida como bucle for, es una herramienta poderosa en la programación que nos permite ejecutar un bloque de código un número específico de veces. Se utiliza comúnmente cuando se conoce de antemano la cantidad de veces que se debe repetir una tarea. La sintaxis típica de un bucle for incluye una variable de control, una condición de salida y un incremento o decremento.

- El bucle for se utiliza para realizar una iteración específica un número determinado de veces.

- Se inicia con la inicialización de una variable de control, seguida de una condición de salida que indica cuándo detener el bucle y, finalmente, una expresión de incremento o decremento que modifica la variable de control en cada iteración.
- Dentro del bucle, se ejecuta el bloque de código tantas veces como se especifica en la condición de salida.

Beneficios del bucle for:

- Control de Iteraciones: Permite ejecutar un bloque de código un número específico de veces.
- Legibilidad del Código: Es una estructura compacta que facilita la comprensión de la lógica de repetición en el código.
- Eficiencia: Es útil cuando se conoce la cantidad exacta de iteraciones necesarias.

Casos de Uso:

- Iteración sobre Rangos Numéricos: Para realizar una tarea un número determinado de veces, como imprimir una secuencia de números o procesar elementos de una lista.
- Recorrido de Estructuras de Datos: Para iterar sobre elementos de listas, matrices o diccionarios.
- Cálculos Repetitivos: Para realizar cálculos repetitivos basados en una fórmula matemática.
- Generación de Secuencias: Para generar secuencias de valores o patrones de datos.

Ejemplo de Uso de bucle for (solo contador):

Supongamos que queremos imprimir los primeros 5 números naturales. Podemos usar un bucle for para controlar la iteración y mostrar los números uno por uno.

```
Algoritmo ejemplo_bucle_for
  Definir i como Entero;

  Para i <- 1 Hasta 5 Con Paso 1 Hacer
    Escribir i;
  FinPara
FinAlgoritmo
```

En este ejemplo, el bucle for se ejecuta desde el valor inicial de i (1) hasta el valor final de i (5), con un paso de 1 en cada iteración. Dentro del bucle, se imprime el valor actual de i. Este bucle imprimirá los números del 1 al 5 en orden ascendente.

```
Algoritmo estructura_repetitiva_for
  Definir i, suma como Entero;

  suma = 0;

  Para i <- 1 Hasta 5 Con Paso 1 Hacer
    suma = suma + i;
  FinPara;

  Escribir "La suma de los números del 1 al 5 es:", suma;
FinAlgoritmo
```

Estructura repetitiva while-do (mientras-hacer).

La estructura repetitiva while-do, también conocida como bucle while, es una herramienta utilizada en programación para ejecutar un bloque de código repetidamente mientras se cumpla una condición específica. Es útil cuando no se conoce de antemano el número exacto de iteraciones que se deben realizar y se necesita que el bucle se ejecute mientras una condición sea verdadera.

- El bucle while comienza evaluando una condición booleana. Si la condición es verdadera, el bloque de código dentro del bucle se ejecuta. Si la condición es falsa, el bucle se detiene y la ejecución continúa después del bucle.
- Es importante garantizar que la condición dentro del bucle eventualmente se vuelva falsa para evitar un bucle infinito.
- El bloque de código dentro del bucle puede modificar las variables que afectan la condición de salida.

Beneficios del bucle while:

- Flexibilidad: Permite ejecutar un bloque de código repetidamente mientras una condición sea verdadera.
- Adaptabilidad: Útil cuando el número de iteraciones no es conocido de antemano o cuando la condición de salida depende de un estado variable.
- Simplicidad: Es una estructura simple y fácil de entender.

Casos de Uso:

- Procesamiento de Entradas del Usuario: Para leer y procesar entradas del usuario hasta que se ingrese un valor específico, como "salir".
- Actualización de Estados: Para actualizar el estado de un programa o una variable hasta que se cumpla una condición de salida.
- Búsqueda de Elementos: Para buscar un elemento específico en una lista o matriz hasta que se encuentre.

- Control de Eventos: Para manejar eventos en tiempo real hasta que se cumpla una condición de terminación.

Ejemplo de Uso de bucle while (solo contador):

Supongamos que queremos imprimir los primeros 5 números naturales. Podemos usar un bucle while para controlar la iteración y mostrar los números uno por uno.

```
Algoritmo ejemplo_bucle_while
  Definir contador como Entero;

  contador = 1; // Inicializamos el contador

  Mientras contador <= 5 Hacer
    Escribir contador;
    contador = contador + 1; // Incrementamos el contador
  FinMientras
FinAlgoritmo
```

En este ejemplo, el bucle while se ejecuta mientras la condición `contador <= 5` sea verdadera. Dentro del bucle, se imprime el valor actual del contador y luego se incrementa en 1 en cada iteración. Este bucle imprimirá los números del 1 al 5 en orden ascendente.

```
Algoritmo estructura_repetitiva_while_do
  Definir contador, suma como Entero;

  contador = 1;
  suma = 0;

  Mientras contador <= 5 Hacer
    suma = suma + contador;
    contador = contador + 1;
  FinMientras;

  Escribir "La suma de los números del 1 al 5 es:", suma;
FinAlgoritmo
```

Estructura repetitiva do-while (hacer-mientras).

La estructura repetitiva do-while, también conocida como bucle do...while, es una herramienta utilizada en programación para ejecutar un bloque de código al menos una vez y luego repetir la ejecución mientras se cumpla una condición específica. A diferencia del bucle while, donde la condición se verifica antes de que se ejecute el bloque de código, en el bucle do-while, el bloque de código se ejecuta primero y luego se verifica la condición.

- La estructura do-while comienza ejecutando un bloque de código una vez sin verificar la condición.
- Después de ejecutar el bloque de código, se verifica la condición. Si la condición es verdadera, el bloque de código se ejecuta nuevamente; de lo contrario, el bucle se detiene.
- Es importante tener en cuenta que el bloque de código se ejecutará al menos una vez, independientemente de si la condición es verdadera o falsa inicialmente.

Beneficios del bucle do-while:

- Garantía de Ejecución: Se asegura de que el bloque de código se ejecute al menos una vez, independientemente de la condición inicial.
- Flexibilidad: Permite ejecutar un bloque de código repetidamente mientras una condición sea verdadera.
- Simplicidad: Es una estructura simple y fácil de entender.

Casos de Uso:

- Validación de Entradas del Usuario: Para solicitar al usuario que ingrese datos y validar las entradas antes de continuar.
- Procesamiento de Menús: Para mostrar un menú al usuario y procesar la selección hasta que se elija una opción de salida.

- Iteración con Acciones Posterior: Cuando se necesita ejecutar una acción y luego verificar si se debe repetir la acción basada en una condición.

Ejemplo de Uso de bucle do-while (solo contador):

Supongamos que queremos imprimir los primeros 5 números naturales. Podemos usar un bucle do-while para controlar la iteración y mostrar los números uno por uno.

```
Algoritmo ejemplo_bucle_do_while
    Definir contador como Entero;

    contador = 1; // Inicializamos el contador

    Hacer
        Escribir contador;
        contador = contador + 1; // Incrementamos el contador
    Mientras contador <= 5;
FinAlgoritmo
```

En este ejemplo, el bucle do-while ejecuta el bloque de código al menos una vez, ya que la condición se verifica al final del bucle. Dentro del bucle, se imprime el valor actual del contador y luego se incrementa en 1 en cada iteración. Este bucle imprimirá los números del 1 al 5 en orden ascendente.

```
Algoritmo estructura_repetitiva_do_while
    Definir contador, suma como Entero;

    contador = 1;
    suma = 0;

    Hacer
        suma = suma + contador;
        contador = contador + 1;
    Mientras contador <= 5;

    Escribir "La suma de los números del 1 al 5 es:", suma;
FinAlgoritmo
```

Salida interna de los bucles

Finalmente, abordaremos la salida interna de los bucles, que nos permite salir de un bucle antes de que se cumpla la condición de salida. Esta técnica se utiliza para optimizar la ejecución de un algoritmo cuando se alcanza un resultado deseado antes de completar todas las iteraciones

Bibliografía y Webgrafía

Steven S. Skiena, Miguel A. Revilla, et ál. Desafíos de programación: El manual de entrenamiento para concursos de programación (Spanish Edition) OJBooks (2020)

Vickler A. Algoritmos. Ladoo Publishing LLC (2022)

Pes C. Pseudocódigo para principiantes. Independently published (27 Abril 2022)

- **Acumulador:** Una variable utilizada para almacenar y sumar valores a lo largo de la ejecución de un programa.
- **Bucle:** Una estructura de control que permite repetir la ejecución de un bloque de código varias veces.
- **Condición de Salida:** Una expresión booleana que determina cuándo debe finalizar la ejecución de un bucle.
- **Contador:** Una variable utilizada para llevar un registro del número de veces que se ha ejecutado un bucle o de la ocurrencia de ciertos eventos en un programa.
- **Do-While:** También conocido como bucle "hacer-mientras", es una estructura de bucle que ejecuta un bloque de código al menos una vez y luego repite la ejecución mientras se cumpla una condición específica.
- **Estructura Algorítmica:** Un conjunto de instrucciones utilizadas en programación para representar la lógica de un algoritmo.
- **For:** También conocido como bucle "desde/para", es una estructura de bucle que permite ejecutar un bloque de código un número específico de veces.
- **Secuencia:** Una estructura algorítmica que especifica el orden en el que se deben ejecutar una serie de instrucciones.
- **While:** También conocido como bucle "mientras-hacer", es una estructura de bucle que ejecuta un bloque de código repetidamente mientras se cumpla una condición específica.